

R Lists Basics

We've covered Vectors, Matrices, and Data frames, now we will learn about our last basic built-in R data structure: the list.

Lists will allow us to store a variety of data structures under a single variable. This means we could store a vector, matrix, data frame, etc. under a single list. For example:

In [19]:

```
# Create a vector
v <- c(1,2,3,4,5)

# Create a matrix
m <- matrix(1:10,nrow=2)

# Create a data frame
df <- women
```

In [20]:

v

Out[20]:

1 2 3 4 5

In [21]:

m

Out[21]:

1	3	5	7	9
2	4	6	8	10

In [22]:

df

Out[22]:

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150

13	height	weight
14	71	159
15	72	164

Using list()

We can use the list() to combine all the data structures:

In [23]:

```
li <- list(v,m,df)
```

In [24]:

```
li
```

Out[24]:

- 1 2 3 4 5
-

1	3	5	7	9
2	4	6	8	10

-

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164

You will notice that the list() assigned numbers to each of the objects in the list, but we can also assign names in the following manner:

In [25]:

```
li <- list(sample_vec = v, sample_mat = m, sample_df = df)
```

In [26]:

```
# Ignore the "error in vapply", this won't occur in RStudio!
li
```

```
Error in vapply(seq_along(mapped), function(i) {: values must be length 1,  
but FUN(X[[3]]) result is length 0
```

Out[26]:

\$sample_vec

1 2 3 4 5

\$sample_mat

1	3	5	7	9
2	4	6	8	10

\$sample_df

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164

Selecting objects from a list

You can use bracket notation to show objects in a list, and double brackets to actually grab the objects from the list, for example:

In [31]:

```
# Single brackets  
li[1] # By index
```

Out[31]:

\$sample_vec =

1 2 3 4 5

In [34]:

```
# By name  
li['sample_vec']
```

Out[34]:

\$sample_vec =

1 2 3 4 5

In [35]:

```
# Notice the type!  
class(li['sample_vec'])
```

Out[35]:

'list'

In [36]:

```
# Use double brackets to actually grab the items  
li[['sample_vec']]
```

Out[36]:

1 2 3 4 5

In [38]:

```
# Can also use $ notation  
li$sample_vec
```

Out[38]:

1 2 3 4 5

You can also index on top of this once you've selected the object from the list, for example:

In [39]:

```
li[['sample_vec']][1] # Second set of indexing
```

Out[39]:

1

In [40]:

```
li[['sample_mat']]
```

Out[40]:

1	3	5	7	9
2	4	6	8	10

In [41]:

```
li[['sample_mat']][1,]
```

Out[41]:

1 3 5 7 9

In [42]:

```
li[['sample_mat']][1:2,1:2]
```

Out[42]:

1	3
2	4

In [43]:

```
li[['sample_df']]['height']
```

Out[43]:

	height
1	58
2	59
3	60
4	61
5	62
6	63
7	64
8	65
9	66
10	67
11	68
12	69
13	70
14	71
15	72

Combining lists

Lists can hold other lists! You can also combine lists using the combine function `c()`:

In [44]:

```
double_list <- c(li,li)
```

In [45]:

```
double_list
```

```
Error in vapply(seq_along(mapped), function(i) {: values must be length 1,  
but FUN(X[[3]]) result is length 0
```

Out[45]:

\$sample_vec

1 2 3 4 5

\$sample_mat

1	3	5	7	9
2	4	6	8	10

\$sample_df

	height	weight
--	--------	--------

1	height	weight
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164

```
$sample_vec
 1 2 3 4 5
```

```
$sample_mat
```

1	3	5	7	9
2	4	6	8	10

```
$sample_df
```

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164

```
In [46]:
```

```
str(double_list)
```

```
List of 6
 $ sample_vec: num [1:5] 1 2 3 4 5
 $ sample_mat: int [1:2, 1:5] 1 2 3 4 5 6 7 8 9 10
 $ sample_df :'data.frame': 15 obs. of  2 variables:
  ..$ height: num [1:15] 58 59 60 61 62 63 64 65 66 67 ...
  ..$ weight: num [1:15] 115 117 120 123 126 129 132 135 139 142 ...
 $ sample_vec: num [1:5] 1 2 3 4 5
 $ sample_mat: int [1:2, 1:5] 1 2 3 4 5 6 7 8 9 10
```

```
$ sample_mat: int [1:2, 1:5] 1 2 3 4 5 6 / 8 9 10
$ sample_df : 'data.frame': 15 obs. of 2 variables:
..$ height: num [1:15] 58 59 60 61 62 63 64 65 66 67 ...
..$ weight: num [1:15] 115 117 120 123 126 129 132 135 139 142 ...
```

That's it for lists and that completes our section on basic data structures!